

Name of the Laboratory: _____

Experiment No: _____ Date: _____

Experiment 1

Installing Cloudera QuickStart VM

Cloudera provides virtual machine (VM) images of complete Apache Hadoop clusters, making it quick and easy to get started with Cloudera CDH.

Note: These are free for personal use, but do require you to register your details on the Cloudera website prior to download. Remember to check your system meets the minimum requirements.

I'm familiar with running VMs...

Go ahead and download the appropriate Cloudera Quickstart VM if you are already familiar with running virtual machines, and have VM software installed. Clone the example files from the FLbigdataD2D GitHub repository once you have Cloudera QuickStart up and running.

I do not have VM software installed...

Follow the instructions to first install VirtualBox (virtual machine client) and then get started with the Cloudera Quickstart VM.

Note: The Cloudera Quickstart VM download file is approx. 5GB, so download time will vary depending on your connection speed. There are online calculators you can use if you'd like an estimate of how long the download may take. Conduct a web search for: how long will my download take.

1. Install Virtual Box (virtual machine client)

a) Download VirtualBox.

Ensure you download the correct version of VirtualBox for your operating system.

PC users: VirtualBox 5.0.14 for Windows hosts (112MB)

MAC users: VirtualBox 5.0.14 for OS X hosts (86.2MB)



Name of the Laboratory: _____

Experiment No: _____ Date: _____

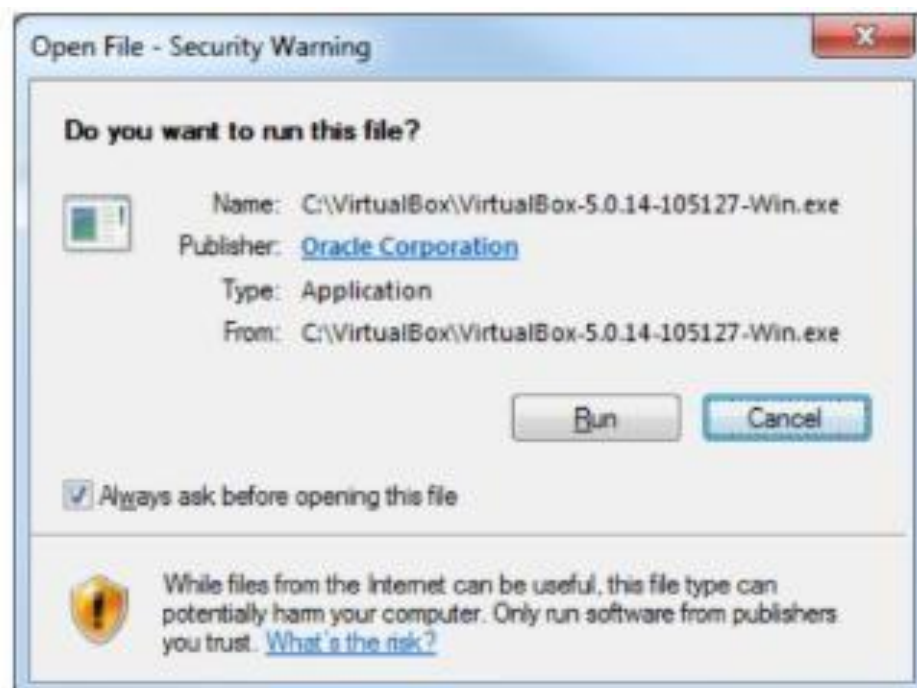
b) Install VirtualBox.

MAC users: Double click the .dmg file you have downloaded and follow the instructions.

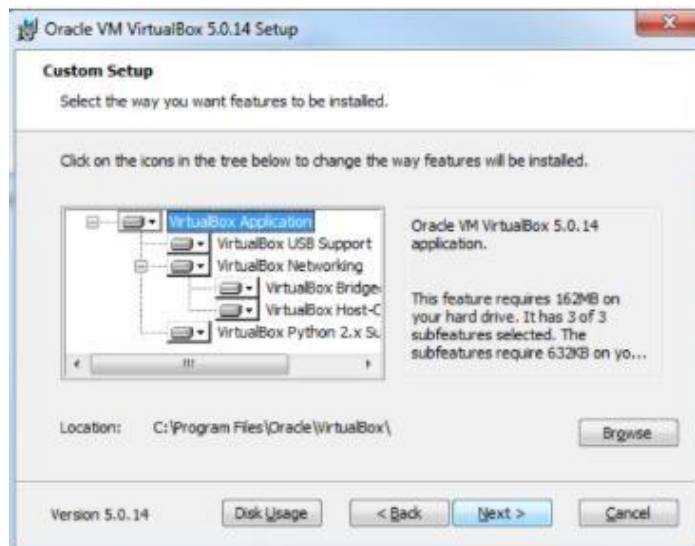


PC users: Double click the .exe file you have downloaded and follow the instructions.

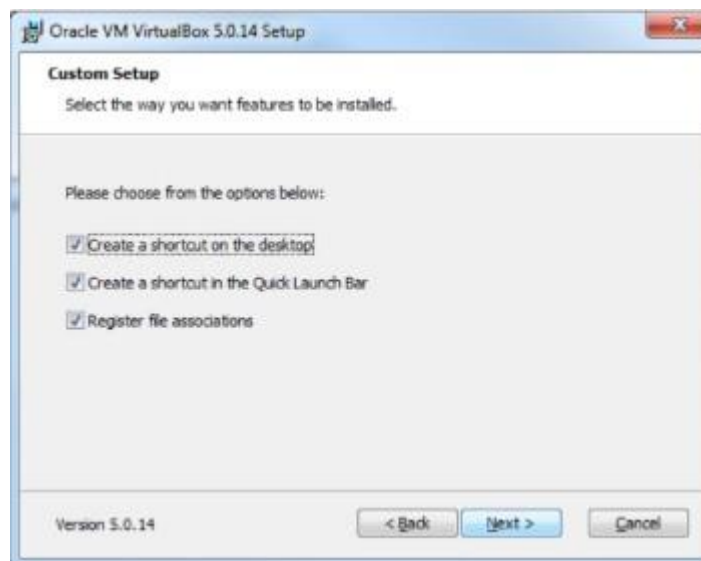
i. Double click .exe file and click Run.



ii. Click Next accepting the suggested custom setup to install features.



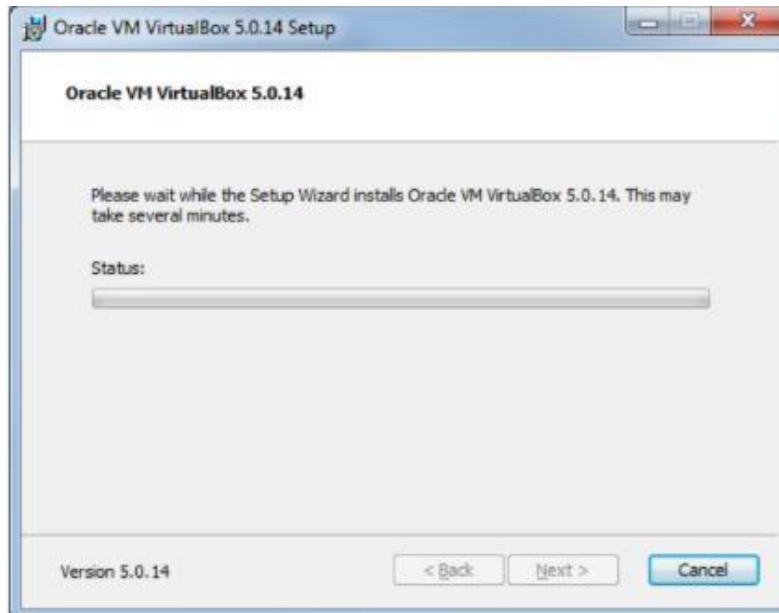
iii. Click Next accepting the suggested custom setup.



iv. Click Yes.

v. Click Install.

vi. Wait for the install to complete. You may receive prompts to confirm the installation as it progresses. Click Yes for each prompt to continue the installation



vii. Click Finish when the installation is complete.

2. Download ClouderaCDH VM for VirtualBox.

- a) Select the virtual machine client you are using from the drop down list (VirtualBox).
- b) Click Download now to begin downloading the installation zip archive file for Cloudera QuickStart VM.
- c) Register and sign in or complete the product interest form.

Before you can download the file you must complete this step. You are required to complete all fields before you can continue. Cloudera may use your details to contact you.



3. Extract files from the Cloudera Quickstart VM zip archive.

Ensure you have sufficient space to extract the archive if you encounter problems.

Alternatively try using 7-zip (Windows) or TheUnArchiver (Mac) which work well with large files.

4. Open the Cloudera Quickstart VM in VirtualBox.

a) Launch VirtualBox.

b) Click on File - Import appliance and select the .ovf file from the location where you have saved the extracted Cloudera Quickstart VM files. Click Next.

Leave 'Reinitialise the MAC address of all network cards' unticked.

c) Click Import.

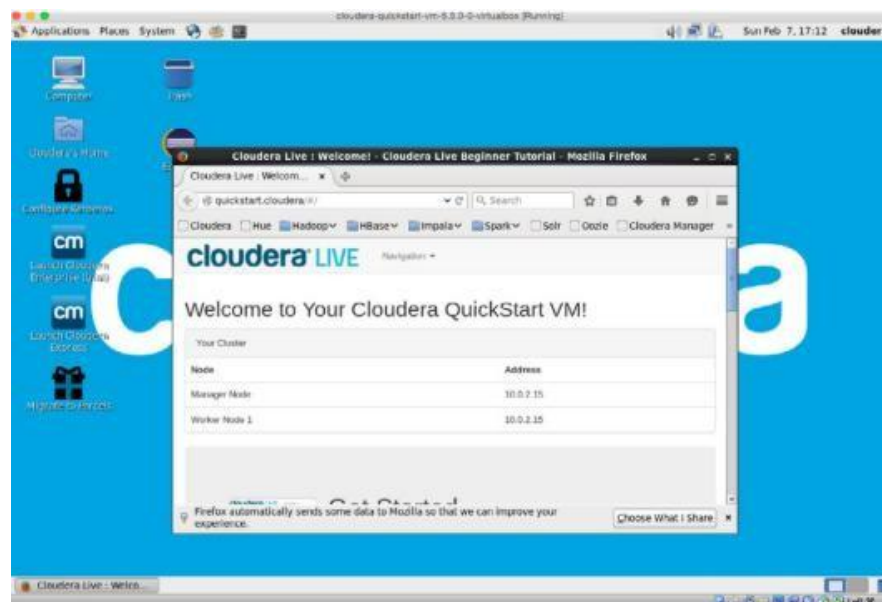
5. Start your Cloudera Quickstart VM instance.

a) Select cloudera-quickstart-vm from the left panel.

b) Click the Start (green arrow) icon.



Note: The VM can take some time to start up. You'll see the blue Cloudera desktop and the Welcome to Cloudera Quickstart VM page once it's ready.



Tip: Once you are finished using Cloudera, remember to shut down the VM by choosing

Name of the Laboratory: _____

Experiment No: _____ Date: _____

System - Shut down.

6. Clone the example files from the FLbigdataD2D GitHub repository.

a) Open the Terminal.

Select Applications > System Tools > Terminal.



b) Change to the Desktop folder.

Type `cd ~/Desktop` in the Terminal window.

c) Clone the example files from GitHub.

Type `git clone https://github.com/QUT-BDA-MOOC/FLbigdataD2D` in the Terminal window.

```
cloudera@quickstart:~/Desktop
File Edit View Search Terminal Help
[cloudera@quickstart ~]$ cd ~/Desktop
[cloudera@quickstart Desktop]$ git clone https://github.com/QUT-BDA-MOOC/FLbigdataD2D
Initialized empty Git repository in /home/cloudera/Desktop/FLbigdataD2D/.git/
remote: Counting objects: 74, done.
remote: Compressing objects: 100% (48/48), done.
remote: Total 74 (delta 22), reused 74 (delta 22), pack-reused 0
Unpacking objects: 100% (74/74), done.
[cloudera@quickstart Desktop]$
```

Name of the Laboratory:

Experiment No: _____ **Date:** _____**LINUX COMMANDS**

These are the most commonly used commands for basic Linux file and directory management.

Directory Management

1. Create a new directory → `mkdir`
2. Create a directory named project → `mkdir project`
3. Create multiple directories at once → `mkdir dir1 dir2 dir3`
4. Create a directory inside another directory → `mkdir parent/child`

Removing Directories

5. Remove an empty directory → `rmdir dirname`
6. Delete a directory along with its contents → `rm -r dirname`

Navigation & Listing

7. Display the current directory → `pwd`
8. List files and directories → `ls`
9. List all files including hidden files → `ls -a`

File Creation & Viewing

10. Create an empty file → `touch filename`
11. Create a file named data.txt → `touch data.txt`
12. View the contents of a file → `cat filename`
13. Write text into a file using a command → `echo "text"; > filename`

File Operations

14. Copy a file → `cp source destination`
15. Rename a file → `mv oldname newname`

Name of the Laboratory: _____

Experiment No: _____ Date: _____

16. Delete a file → `rm filename`

17. Move a file from one directory to another → `mv filename`

`/path/to/directory`

Permissions & Size

18. Check file permissions → `ls -l filename`

19. Change file permissions → `chmod permissions filename`

Example: `chmod 755 filename`

20. Show the size of a file → `ls -lh filename`

Basic UNIX Commands to Create a Directory And File

1. `pwd` //Present Working Directory

2. `whoami` //which user account is currently active

3. `mkdir <directory name>` //to create a directory

4. `ls` //to check list of files and directories

5. `cd firstdir` //change directory

6. `cd ..` //change to parent directory

7. `touch <file name>` //creates an empty file

8. `vi <filename>` //to view,edit a file

Common vi command in linux

- `i` → insert text
- `Esc` → return to command mode
- `:w` → save file
- `:q` → quit
- `:wq` → save and quit
- `:q!` → quit without saving

9. `cat <filename>` //display the content of a file

Name of the Laboratory: _____**Experiment No:** _____ **Date:** _____**1. HDFS (Storage)**

A. Hadoop Storage File system: Your first objective is to create a directory structure in HDFS using HDFS commands. Create the local files using Linux commands and move the files to HDFS directory and vice versa.

I. Write a command to create the directory structure in HDFS.

II. Write a Command to move file from local unix/linux machine to HDFS.

B. Viewing Data Contents, Files and Directory Try to perform these simple steps:

Write HDFS command to see the contents of files in HDFS.

C. Getting Files data from the HDFS to Local Disk

I. Write a HDFS command to copy the file from HDFS to local file system. To process any data first move the data in HDFS. All files stored in HDFS can be accessed using HDFS commands.

Ans) A)

I) [cloudera@quickstart ~]\$ hadoop fs -mkdir Directory

**II) [cloudera@quickstart ~]\$ hadoop fs -put /home/cloudera/ Desktop/myfile.txt
Directory**

[cloudera@quickstart ~]\$ hadoop fs -ls

Directory O/P:-

Found 1 items

-rw-r--r-- 1 cloudera cloudera
Directory/myfile.txt

14 2023-05-02 01:36

**B) [cloudera@quickstart ~]\$ hadoop fs -cat
Directory/myfile.txt Hello, World!**

**C) [cloudera@quickstart ~]\$ hadoop fs -get Directory/myfile.txt
/home/cloudera/Desktop/myfile1.txt**

Experiment 2

A. Develop MapReduce example program in a MapReduce environment to find out the number of occurrences of each word in a text file

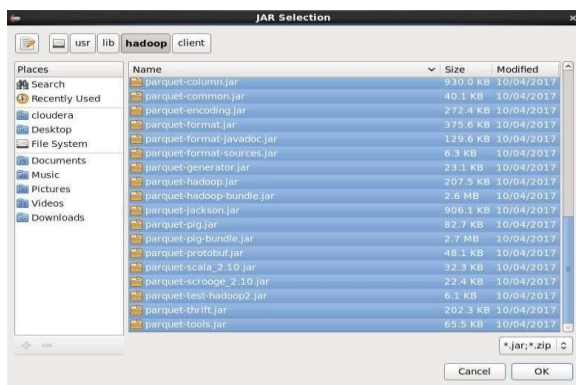
1 .Create a new java project .Give name as WordCount.

Then click next

2.then go to libraries there click on add external jar

files then File system->usr->lib->hadoop

Copy all the jar files

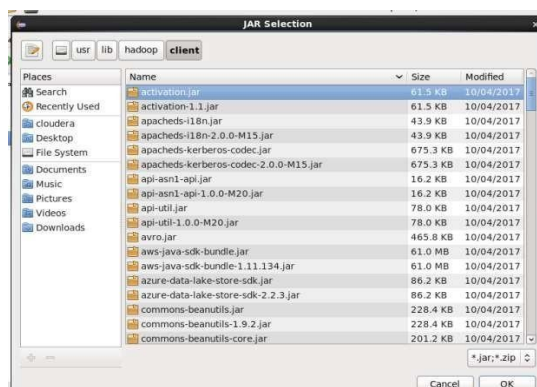


Then click on ok.

3. Again click on add external jar

files, goto File system->usr-

>lib->hadoop->client



Name of the Laboratory: _____**Experiment No:** _____ **Date:** _____

Select all the files and then click ok.

4. click on finish

5. Now click on wordCount->src create a new class WordCount.java

From apache documentation of hadoop goto MapReduce tutorials copy the java code

Java code:

```
import
java.io.IOException;
import
java.util.StringTokenizer;

import
org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import
org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import
org.apache.hadoop.mapreduce.Job;
import
org.apache.hadoop.mapreduce.Mapper;
import
org.apache.hadoop.mapreduce.Reducer;
import
org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import
org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class WordCount {

    public static class TokenizerMapper
        extends Mapper<Object, Text, Text, IntWritable>{
```

Name of the Laboratory: _____**Experiment No:** _____ **Date:** _____

```
private final static IntWritable one = new
IntWritable(1); private Text word = new Text();

public void map(Object key, Text value, Context context
    ) throws IOException, InterruptedException {
    StringTokenizer itr = new
    StringTokenizer(value.toString()); while
    (itr.hasMoreTokens()) {
        word.set(itr.nextToke
        n());
        context.write(word,
        one);
    }
}

}

}

public static class IntSumReducer
    extends
    Reducer<Text,IntWritable,Text,IntWritable> {
    private IntWritable result = new IntWritable();

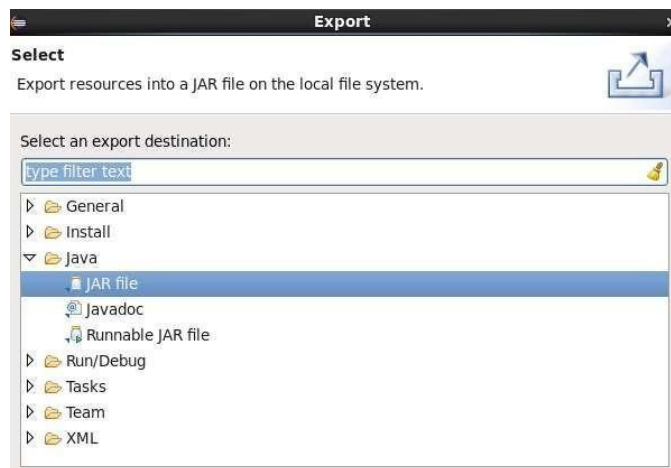
    public void reduce(Text key, Iterable<IntWritable>
        values, Context context
        ) throws IOException,
        InterruptedException { int sum = 0;
        for (IntWritable val :
            values) { sum +=
            val.get();
        }

        result.set(sum);
        context.write(key,
        result);
    }
}
```

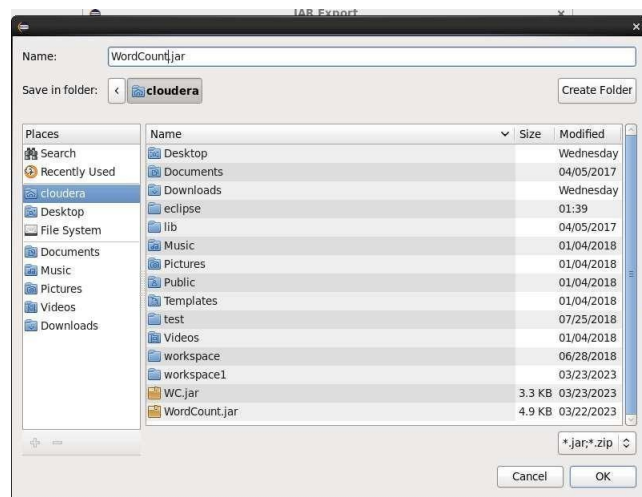
```
public static void main(String[] args) throws Exception {  
    Configuration conf = new Configuration();  
    Job job = Job.getInstance(conf, "word count");  
    job.setJarByClass(WordCount.class);  
    job.setMapperClass(TokenizerMapper.class);  
    job.setCombinerClass(IntSumReducer.class);  
    job.setReducerClass(IntSumReducer.class);  
    job.setOutputKeyClass(Text.class);  
    job.setOutputValueClass(IntWritable.class);  
    FileInputFormat.addInputPath(job, new Path(args[0]));  
    FileOutputFormat.setOutputPath(job, new Path(args[1]));  
    System.exit(job.waitForCompletion(true) ? 0 : 1);  
}  
}
```

6. save the java code and run it.

7. Right click on WordCount folder->export->select jar file->click on next



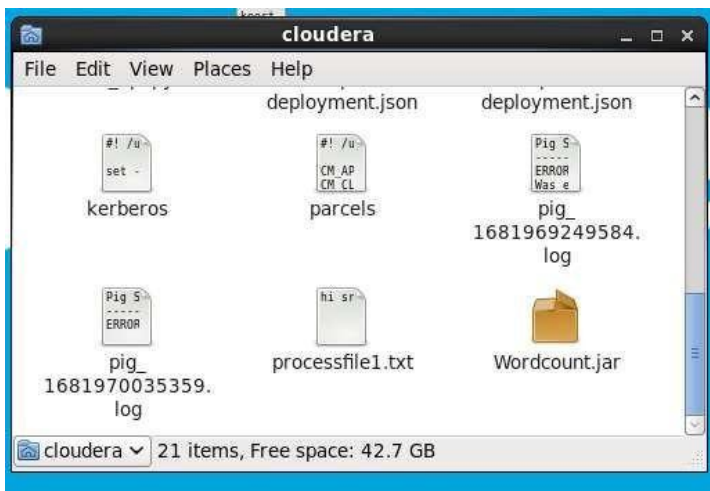
8. Name the jar file and select the required location the jar file to be exported -
>ok->finish



Select the export destination:

JAR file:

1. Go to the location where we exported the jar file and verify it.



2. Now open terminal and do the following operations

```
[cloudera@quickstart ~]$ pwd
/home/cloudera
[cloudera@quickstart ~]$ ls
cloudera-manager  eclipse  Music  Videos
cm_api.py         enterprise-deployment.json  parcels  Wordcount.jar
Desktop          express-deployment.json   Pictures  workspace
Documents        kerberos  Public
Downloads        lib       Templates
```

Name of the Laboratory: _____**Experiment No:** _____ **Date:** _____

Check for wordcount.jar file

```
[cloudera@quickstart ~]$ cat >/home/cloudera/processfile1.txt  
hi sravanthi hello keerthana
```

```
[cloudera@quickstart ~]$ hdfs dfs -mkdir /input  
[cloudera@quickstart ~]$ hadoop dfs -put /home/cloudera/Desktop/processfile.txt  
/input/processfile1.txt
```

```
[cloudera@quickstart ~]$ hdfs dfs -cat /input/processfile1.txt  
hello sravanthi  
keerthana hi hi  
hi sravanthi  
hello
```

```
[cloudera@quickstart ~]$ hadoop jar /home/cloudera/Wordcount.jar WordCount  
/input/processfile1.txt /out
```

```
[cloudera@quickstart ~]$ hdfs dfs -ls /out  
Found 2 items  
-rw-r--r--  1 cloudera supergroup      0 2023-04-12 22:05 /out/_SUCCESS  
-rw-r--r--  1 cloudera supergroup    37 2023-04-12 22:05 /out/part-r-00000
```

Check the number /out/part-r-00000

```
[cloudera@quickstart ~]$ hdfs dfs -cat /out/part-r-00000
```

The final output will be

```
hello  2  
hi      3  
keerthana  1  
sravanthi  2
```


Name of the Laboratory:

Experiment No: _____ **Date:** _____**Experiment 3**

3. Data Processing Tool – Hive (NOSQL query based language) Hive command line tool allows you to submit jobs via bash scripts.

Identifying properties of a data set:

We have a table 'user data' that contains the following

fields: data_date: string

user_id:

string

properties

: string

The properties field is formatted as a series of attribute=value

pairs. Ex: Age=21; state=CA; gender=M;

Ans)

I. Create the table in HIVE using hive nosql based query.

```
hive> CREATE TABLE IF NOT EXISTS user_data1 ( data_date
string, user_id string, properties Map<string,string> )
> ROW FORMAT DELIMITED
> FIELDS TERMINATED BY ','
> COLLECTION ITEMS TERMINATED BY '#'
> MAP KEYS TERMINATED BY '@'
> STORED AS TEXTFILE;
```

OK

Time taken: 0.091 seconds

II. Fill the table with sample data by using some sample data bases.

Create a text file with dummy data and load it using the following command.

Name of the Laboratory: _____**Experiment No:** _____ **Date:** _____

Dummy Data:

```
22-04-2023,1,Age@21#State@CA#Gender@M
23-04-2023,2,Age@21#State@NY#Gender@F
24-04-2023,3,Age@22#State@OH#Gender@F
25-04-2023,4,Age@22#State@OH#Gender@M
26-04-2023,5,Age@23#State@TX#Gender@F
27-04-2023,6,Age@23#State@TN#Gender@M
```

```
hive> LOAD DATA INPATH 'Directory/user_database.txt' overwrite
into table user_data1;
```

Loading data to table

```
default.user_data1 chgrp: changing
ownership of
```

```
'hdfs://quickstart.cloudera:8020/user/hive/warehouse/user_data1/user_d
atab ase.txt': User does not belong to supergroup
```

```
Table default.user_data1 stats: [numFiles=1, numRows=0,
totalSize=234, rawDataSize=0]
```

OK

Time taken: 0.699 seconds

```
hive> select * from
```

```
user_data1; OK
```

```
22-04-2023      1
```

```
  {"Age":"21","State":"CA","Gender":"M","":null}
```

```
23-04-2023      2
```

```
  {"Age":"21","State":"NY","Gender":"F","":null}
```

```
24-04-2023      3
```

```
  {"Age":"22","State":"OH","Gender":"F","":null}
```

```
25-04-2023      4
```

```
  {"Age":"22","State":"OH","Gender":"M","":null}
```

```
26-04-2023      5
```

```
  {"Age":"23","State":"TX","Gender":"F","":null}
```

```
27-04-2023      6
```

```
  {"Age":"23","State":"TN","Gender":"M","":null}
```

```
Time taken: 0.394 seconds, Fetched: 6 row(s)
```

Name of the Laboratory: _____**Experiment No:** _____ **Date:** _____

- III. Write a program that produces a list of properties with minimum value(min_value), largest value(max_value) and number of unique values. Before you start, execute the prepare step to load the data into HDFS.

IV.

Minimum values:

```
hive> select t.my_key, min(t.my_key_value) as min_value
> from (
> select explode(properties) as (my_key,my_key_value)
> from user_data1
> ) t
> group by t.my_key;
```

...

Age 21

Gender F

State CA

Time taken: 26.099 seconds, Fetched: 3 row(s)

Maximum values:

```
hive> select t.my_key, max(t.my_key_value) as max_value
> from (
> select explode(properties) as (my_key,my_key_value)
> from user_data1
> ) t
> group by t.my_key;
```

...

OK

Age 23

Gender M

State TX

Time taken: 22.254 seconds, Fetched: 3 row(s)

Unique Coun

```
hive> select tmy_key, count(distinct t.my_key_value) as unique_count
> from (
> select explode(properties) as (my_key,my_key_value)
```

Name of the Laboratory:

Experiment No:_____ **Date:** _____

```
> from user_data1
> ) t
> group by t.my_key;
...
```

```
Age      3
Gender   2
State    5
```

Time taken: 19.956 seconds, Fetched: 3 row(s)

V. Generate a count per state.

```
hive> select my_value, count(*) as state_count
> from (
> select explode(properties) as (my_state, my_value)
> from user_data1
> ) t
> where t.my_state = 'State'
> group by t.my_value;
...
```

```
CA      1
NY      1
OH      2
TN      1
TX      1
```

Time taken: 21.772 seconds, Fetched: 5 row(s)

Name of the Laboratory: _____**Experiment No:** _____ **Date:** _____**Experiment 4**

Data Processing Tool – Pig (Latin based scripting lang)

Pig command line tool like the Hive allows you to submit jobs via bash scripts.

A) Simple Logs

We have a set of log files and need to create a job that runs every hour and perform some calculations.

The log files are delimited by a 'tab' character and have the following fields:

- a) site
- b) hour_of_day
- c) page_views
- d) data_date

The log files are located on the prepare folder. Load them in HDFS at data/ pig/ simple_logs folder and use them as the input.

Important: In order to load tab delimited files use pigStorage ('\u0001').

Lab Instructions:

Create a program to:

- I. Calculate the total views per hour per day.
- II. Calculate the total views per day.
- III. Calculate the total counts of each hour across all days.
- IV. We can write word count script by passing text file as input

Ans) Create a text file with dummy data.

Dummy data:

```
"site1",1,10,"2023-04-27"  
"site2",1,5,"2023-04-27"  
"site1",2,15,"2023-04-27"  
"site2",2,20,"2023-04-27"  
"site1",1,5,"2023-04-28"  
"site2",1,10,"2023-04-28"  
"site1",2,25,"2023-04-28"  
"site2",2,30,"2023-04-28"
```

Name of the Laboratory: _____**Experiment No:** _____ **Date:** _____

i, ii & iii)

Create a pig script file with following commands and execute it.

-- Load the log data from HDFS

```
logs = LOAD '/home/cloudera/sample_logs.txt' USING PigStorage(',') AS  
(site:chararray, hour:int, views:int, date:chararray);
```

-- Calculate total views per hour per day

```
hourly_views = GROUP logs BY (date, hour);
```

```
hourly_views_sum = FOREACH hourly_views GENERATE group.date  
AS date, group.hour AS hour, SUM(logs.views) AS views;
```

-- Calculate total views per day

```
daily_views = GROUP logs BY
```

```
date;
```

```
daily_views_sum = FOREACH daily_views GENERATE group AS date,  
SUM(logs.views) AS views;
```

-- Calculate the total counts of each hour across all days

```
hour_counts = GROUP logs BY hour;
```

```
hour_counts_sum = FOREACH hour_counts GENERATE group AS hour,  
SUM(logs.views) AS views;
```

-- Store the results in HDFS

```
STORE hourly_views_sum INTO '/home/cloudera/hourly_views' USING  
PigStorage(',');
```

```
STORE daily_views_sum INTO '/home/cloudera/daily_views' USING  
PigStorage(',');
```

```
STORE hour_counts_sum INTO '/home/cloudera/hour_counts' USING  
PigStorage(',');
```

Name of the Laboratory: _____**Experiment No:** _____ **Date:** _____

O/P:- It is stored at the given path, i.e., /home/cloudera. The output is produced as a folder containing a text file. The text file has the expected answer.

1) "2023-04-27",1,15

"2023-04-27",2,35

"2023-04-28",1,15

"2023-04-28",2,55

2)1

,30

2,90

3)"2023-04-27",50

"2023-04-28",70

4)Create a pig script file with following commands and execute it.

Give some text file's path as input instead of /home/cloudera/myfile.txt.

```
data = LOAD '/home/cloudera/myfile.txt' AS (line:chararray);
words = FOREACH data GENERATE FLATTEN(TOKENIZE(line)) AS
word;
word_counts = GROUP words BY word;
word_count_totals = FOREACH word_counts GENERATE group AS word,
COUNT(words) AS count;
STORE word_count_totals INTO '/home/cloudera';
```

O/P:- It is stored at the given path, i.e., /home/cloudera. The output is produced as a folder containing a text file. The text file has the expected answer.

Hello 1

World! 1

VNRVJIET

Name of the Experiment: _____

Name of the Laboratory:

Experiment No:_____ Date: _____

-- to get grunt cell command is

```
[cloudera@quickstart ~]$ pig -x local
```

```
clouda-quickstart-v5.13.0-0-virtualbox [Running] - Oracle VM VirtualBox
File Machine View Input Devices Help
Applications Places System
Sun Feb 22, 10:40 PM cloudera
cloudera@quickstart:~$
File Edit View Search Terminal Help

Logging initialized using configuration in file:/etc/hive/conf.dist/hive-log4j.properties
WARNING: Hive CLI is deprecated and migration to Beeline is recommended.
hive> show databases
;
> ;
OK
bigdata
default
practice
Time taken: 0.709 seconds, Fetched: 3 row(s)
hive>
[1]+  Stopped                  hive
[cloudera@quickstart ~]$ pig -x local
log4j:WARN No appenders could be found for logger (org.apache.hadoop.util.Shell).
log4j:WARN Please initialize the log4j system properly.
log4j:WARN See http://logging.apache.org/log4j/1.2/faq.html#noconfig for more info.
2026-02-22 22:40:39,215 [main] INFO org.apache.pig.Main - Apache Pig version 0.12.0-cdh5.13.0 (reexported) compiled Oct 04 2017, 11:09:03
2026-02-22 22:40:39,215 [main] INFO org.apache.pig.Main - Logging error messages to: /home/cloudera/pig/1771820839202.log
2026-02-22 22:40:39,301 [main] INFO org.apache.hadoop.mapreduce.lib.input.FileInputFormat - file:/home/cloudera/pigbootstrap not found
2026-02-22 22:40:39,386 [main] INFO org.apache.hadoop.conf.Configuration.deprecation - fs.default.name is deprecated. Instead, use fs.defaultFS
2026-02-22 22:40:39,386 [main] INFO org.apache.hadoop.conf.Configuration.deprecation - mapred.job.tracker is deprecated. Instead, use mapreduce.jobtracker.address
2026-02-22 22:40:39,387 [main] INFO org.apache.hadoop.mapreduce.executionengine.HExecutionEngine - Connecting to hadoop file system at: file:///
2026-02-22 22:40:39,713 [main] INFO org.apache.hadoop.conf.Configuration.deprecation - io.bytes.per.checksum is deprecated. Instead, use dfs.bytes-per-checksum
2026-02-22 22:40:39,811 [main] INFO org.apache.hadoop.conf.Configuration.deprecation - fs.default.name is deprecated. Instead, use fs.defaultFS
2026-02-22 22:40:39,812 [main] INFO org.apache.hadoop.conf.Configuration.deprecation - mapred.job.tracker is deprecated. Instead, use mapreduce.jobtracker.address
2026-02-22 22:40:39,818 [main] INFO org.apache.hadoop.conf.Configuration.deprecation - io.bytes.per.checksum is deprecated. Instead, use dfs.bytes-per-checksum
2026-02-22 22:40:39,913 [main] INFO org.apache.hadoop.conf.Configuration.deprecation - fs.default.name is deprecated. Instead, use fs.defaultFS
2026-02-22 22:40:39,913 [main] INFO org.apache.hadoop.conf.Configuration.deprecation - mapred.job.tracker is deprecated. Instead, use mapreduce.jobtracker.address
2026-02-22 22:40:39,914 [main] INFO org.apache.hadoop.conf.Configuration.deprecation - io.bytes.per.checksum is deprecated. Instead, use dfs.bytes-per-checksum
2026-02-22 22:40:39,983 [main] INFO org.apache.hadoop.conf.Configuration.deprecation - fs.default.name is deprecated. Instead, use fs.defaultFS
2026-02-22 22:40:40,006 [main] INFO org.apache.hadoop.conf.Configuration.deprecation - mapred.job.tracker is deprecated. Instead, use mapreduce.jobtracker.address
2026-02-22 22:40:40,006 [main] INFO org.apache.hadoop.conf.Configuration.deprecation - io.bytes.per.checksum is deprecated. Instead, use dfs.bytes-per-checksum
2026-02-22 22:40:40,083 [main] INFO org.apache.hadoop.conf.Configuration.deprecation - fs.default.name is deprecated. Instead, use fs.defaultFS
2026-02-22 22:40:40,084 [main] INFO org.apache.hadoop.conf.Configuration.deprecation - mapred.job.tracker is deprecated. Instead, use mapreduce.jobtracker.address
2026-02-22 22:40:40,093 [main] INFO org.apache.hadoop.conf.Configuration.deprecation - io.bytes.per.checksum is deprecated. Instead, use dfs.bytes-per-checksum
2026-02-22 22:40:40,174 [main] INFO org.apache.hadoop.conf.Configuration.deprecation - fs.default.name is deprecated. Instead, use fs.defaultFS
2026-02-22 22:40:40,174 [main] INFO org.apache.hadoop.conf.Configuration.deprecation - mapred.job.tracker is deprecated. Instead, use mapreduce.jobtracker.address
2026-02-22 22:40:40,175 [main] INFO org.apache.hadoop.conf.Configuration.deprecation - io.bytes.per.checksum is deprecated. Instead, use dfs.bytes-per-checksum
2026-02-22 22:40:40,250 [main] INFO org.apache.hadoop.conf.Configuration.deprecation - fs.default.name is deprecated. Instead, use fs.defaultFS
2026-02-22 22:40:40,250 [main] INFO org.apache.hadoop.conf.Configuration.deprecation - mapred.job.tracker is deprecated. Instead, use mapreduce.jobtracker.address
2026-02-22 22:40:40,250 [main] INFO org.apache.hadoop.conf.Configuration.deprecation - io.bytes.per.checksum is deprecated. Instead, use dfs.bytes-per-checksum
2026-02-22 22:40:40,251 [main] INFO org.apache.hadoop.conf.Configuration.deprecation - fs.default.name is deprecated. Instead, use fs.defaultFS
2026-02-22 22:40:40,311 [main] INFO org.apache.hadoop.conf.Configuration.deprecation - mapred.job.tracker is deprecated. Instead, use mapreduce.jobtracker.address
2026-02-22 22:40:40,311 [main] INFO org.apache.hadoop.conf.Configuration.deprecation - io.bytes.per.checksum is deprecated. Instead, use dfs.bytes-per-checksum
2026-02-22 22:40:40,312 [main] INFO org.apache.hadoop.conf.Configuration.deprecation - fs.default.name is deprecated. Instead, use fs.defaultFS
2026-02-22 22:40:40,366 [main] INFO org.apache.hadoop.conf.Configuration.deprecation - mapred.job.tracker is deprecated. Instead, use mapreduce.jobtracker.address
2026-02-22 22:40:40,366 [main] INFO org.apache.hadoop.conf.Configuration.deprecation - io.bytes.per.checksum is deprecated. Instead, use dfs.bytes-per-checksum
2026-02-22 22:40:40,366 [main] INFO org.apache.hadoop.conf.Configuration.deprecation - io.bytes.per.checksum is deprecated. Instead, use dfs.bytes-per-checksum
grant$
```

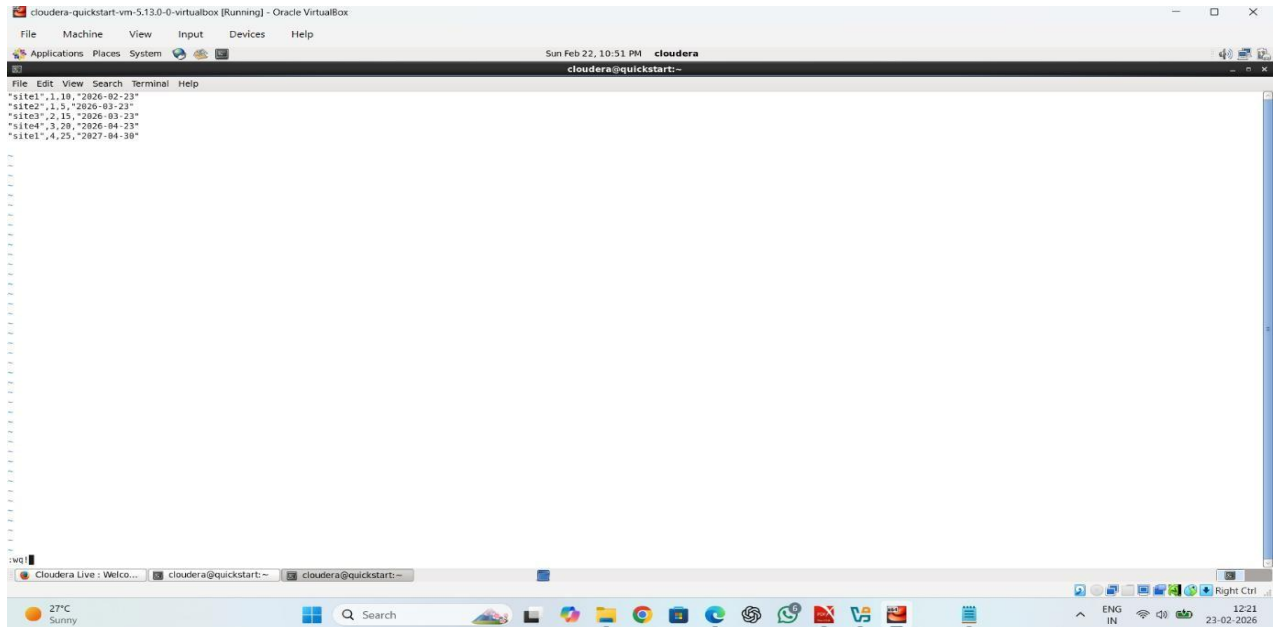
```
grunt>
```

--command to see file list

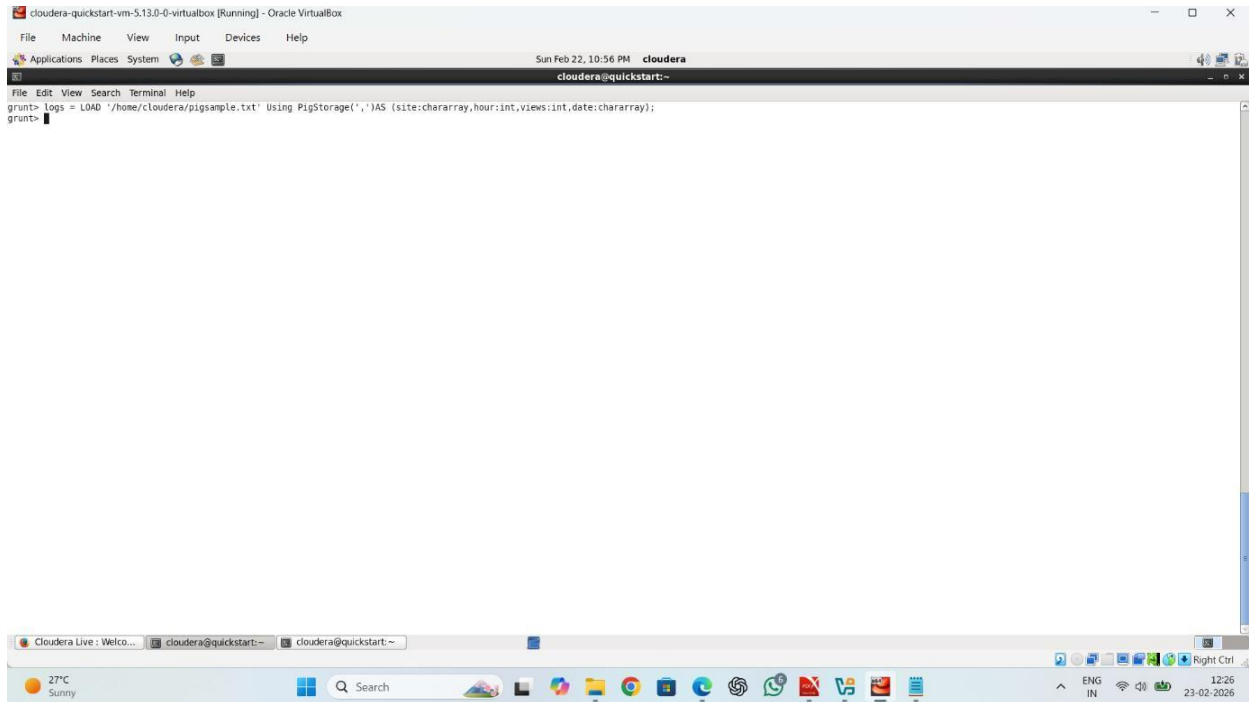
```
grunt> fs -ls
```

```
Oscara quickstart em-5.13.0-venice [Running] | Oracle VM VirtualBox  
File Machine View Input Devices Help  
Applications Places System  
Sun Feb 22, 18:42 PM cloudera  
cloudera@quickstart:-  
Browse and run installed applications  
File Edit View Search Terminal Help  
2026-02-22 22:41:27,979 [main] INFO org.apache.hadoop.conf.Configuration.deprecation - fs.default.name is deprecated. Instead, use fs.defaultFS  
2026-02-22 22:41:27,879 [main] INFO org.apache.hadoop.conf.Configuration.deprecation - mapred.job.tracker is deprecated. Instead, use mapreduce.jobtracker.address  
2026-02-22 22:41:28,000 [main] INFO org.apache.hadoop.mapreduce.v2.util.DfsUtils$DfsConfiguration - io.bytes.per.checksum is deprecated. Instead, use dfs.bytes-per-checksum  
2026-02-22 22:41:28,049 [main] INFO org.apache.hadoop.conf.Configuration.deprecation - fs.default.name is deprecated. Instead, use fs.defaultFS  
2026-02-22 22:41:28,050 [main] INFO org.apache.hadoop.mapreduce.v2.util.DfsUtils$DfsConfiguration - mapred.job.tracker is deprecated. Instead, use mapreduce.jobtracker.address  
2026-02-22 22:41:28,094 [main] INFO org.apache.hadoop.conf.Configuration.deprecation - io.bytes.per.checksum is deprecated. Instead, use dfs.bytes-per-checksum  
2026-02-22 22:41:28,092 [main] INFO org.apache.hadoop.conf.Configuration.deprecation - fs.default.name is deprecated. Instead, use fs.defaultFS  
2026-02-22 22:41:28,133 [main] INFO org.apache.hadoop.mapreduce.v2.util.DfsUtils$DfsConfiguration - mapred.job.tracker is deprecated. Instead, use mapreduce.jobtracker.address  
2026-02-22 22:41:28,183 [main] INFO org.apache.hadoop.conf.Configuration.deprecation - io.bytes.per.checksum is deprecated. Instead, use dfs.bytes-per-checksum  
grants  
grant fs fs:  
permissions: fs.defaultFS is not set when running '-ls' command.  
Found 49 items  
drwxr-xr-x 1 cloudera cloudera 4096 2026-02-16 21:05 .ifconfighistory  
-rw-r--r-- 1 cloudera cloudera 135 2026-02-02 28:58 .bash_history  
-rw-r--r-- 1 cloudera cloudera 18 2017-10-23 09:14 .ssh_login  
-rw-r--r-- 1 cloudera cloudera 176 2017-10-23 09:14 .bash_profile  
-rw-r--r-- 1 cloudera cloudera 176 2017-10-23 09:14 .bashrc  
dwxr-xr-x 1 cloudera cloudera 4096 2026-01-24 01:04 .cache  
dwxr-xr-x 1 cloudera cloudera 4096 2026-01-24 01:04 .config  
-rw-r--r-- 1 cloudera cloudera 4096 2026-01-24 01:04 .dmcc  
-rw-r--r-- 1 cloudera cloudera 16 2026-01-24 01:04 .and_auth  
dwxr-xr-x 1 cloudera cloudera 4096 2026-01-24 01:04 .fontconfig  
-rw-r--r-- 1 cloudera cloudera 4096 2026-02-16 21:05 .gnome  
-rw-r--r-- 1 cloudera cloudera 4096 2026-02-22 22:08 .gconfd  
-rw-r--r-- 1 cloudera cloudera 4096 2026-01-24 01:04 .gnome2  
dwxr-xr-x 1 cloudera cloudera 4096 2026-01-24 01:04 .gnome-private  
-rw-r--r-- 1 cloudera cloudera 4096 2026-01-24 01:05 .gtksource3-1.0  
-rw-r--r-- 1 cloudera cloudera 152 2026-02-16 21:05 .gtk_bookmarks  
-rw-r--r-- 1 cloudera cloudera 4096 2026-01-24 01:04 .gvfs  
dwxr-xr-x 1 cloudera cloudera 4096 2026-01-24 01:04 .local  
dwxr-xr-x 1 cloudera cloudera 4096 2017-10-23 15:55 .mozilla  
dwxr-xr-x 1 cloudera cloudera 4096 2026-01-24 01:04 .mullins  
-rw-r--r-- 1 cloudera cloudera 7 2026-02-22 22:41 .pig_history  
-rw-r--r-- 1 cloudera cloudera 4096 2026-01-24 01:04 .polite  
-rw-r--r-- 1 cloudera cloudera 256 2026-01-24 01:04 .polite-cookie  
-rw-r--r-- 1 cloudera cloudera 2026-02-16 21:05 .vscodecli-cloudword.pid  
-rw-r--r-- 1 cloudera cloudera 5 2026-02-16 21:05 .vscodecli-display.pid  
-rw-r--r-- 1 cloudera cloudera 5 2026-02-16 21:05 .vscodecli-dragdrop.pid  
-rw-r--r-- 1 cloudera cloudera 5 2026-02-16 21:05 .vscodecli-sentinel.pid  
-rw-r--r-- 1 cloudera cloudera 1927 2026-02-02 21:06 .viminfo  
-rw-r--r-- 1 cloudera cloudera 877 2026-02-16 21:18 .xsession-errors  
-rw-r--r-- 1 cloudera cloudera 2229 2026-02-02 28:22 .xsession-errors.old  
dwxr-xr-x 1 cloudera cloudera 4096 2017-10-23 15:55 Desktop  
dwxr-xr-x 1 cloudera cloudera 4096 2017-10-23 15:55 Documents  
dwxr-xr-x 1 cloudera cloudera 4096 2026-01-24 01:04 Downloads  
dwxr-xr-x 1 cloudera cloudera 4096 2026-01-24 01:04 Music  
dwxr-xr-x 1 cloudera cloudera 4096 2026-01-24 01:04 Pictures
```


--load file



```
cloudera-quickstart-vm-5.13.0-0-virtualbox (Running) - Oracle VirtualBox
File Machine View Input Devices Help
Sun Feb 22, 10:51 PM cloudera
cloudera@quickstart:~
File Edit View Search Terminal Help
"site1",1,18,"2026-02-23"
"site2",1,5,"2026-03-23"
"site3",2,15,"2026-03-23"
"site4",3,20,"2026-04-23"
"site5",4,25,"2027-04-30"
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
101
102
103
104
105
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000
1001
1002
1003
1004
1005
1006
1007
1008
1009
1010
1011
1012
1013
1014
1015
1016
1017
1018
1019
1020
1021
1022
1023
1024
1025
1026
1027
1028
1029
1030
1031
1032
1033
1034
1035
1036
1037
1038
1039
1040
1041
1042
1043
1044
1045
1046
1047
1048
1049
1050
1051
1052
1053
1054
1055
1056
1057
1058
1059
1060
1061
1062
1063
1064
1065
1066
1067
1068
1069
1070
1071
1072
1073
1074
1075
1076
1077
1078
1079
1080
1081
1082
1083
1084
1085
1086
1087
1088
1089
1090
1091
1092
1093
1094
1095
1096
1097
1098
1099
1100
1101
1102
1103
1104
1105
1106
1107
1108
1109
1110
1111
1112
1113
1114
1115
1116
1117
1118
1119
1120
1121
1122
1123
1124
1125
1126
1127
1128
1129
1130
1131
1132
1133
1134
1135
1136
1137
1138
1139
1140
1141
1142
1143
1144
1145
1146
1147
1148
1149
1150
1151
1152
1153
1154
1155
1156
1157
1158
1159
1160
1161
1162
1163
1164
1165
1166
1167
1168
1169
1170
1171
1172
1173
1174
1175
1176
1177
1178
1179
1180
1181
1182
1183
1184
1185
1186
1187
1188
1189
1190
1191
1192
1193
1194
1195
1196
1197
1198
1199
1200
1201
1202
1203
1204
1205
1206
1207
1208
1209
1210
1211
1212
1213
1214
1215
1216
1217
1218
1219
1220
1221
1222
1223
1224
1225
1226
1227
1228
1229
1230
1231
1232
1233
1234
1235
1236
1237
1238
1239
1240
1241
1242
1243
1244
1245
1246
1247
1248
1249
1250
1251
1252
1253
1254
1255
1256
1257
1258
1259
1260
1261
1262
1263
1264
1265
1266
1267
1268
1269
1270
1271
1272
1273
1274
1275
1276
1277
1278
1279
1280
1281
1282
1283
1284
1285
1286
1287
1288
1289
1290
1291
1292
1293
1294
1295
1296
1297
1298
1299
1300
1301
1302
1303
1304
1305
1306
1307
1308
1309
1310
1311
1312
1313
1314
1315
1316
1317
1318
1319
1320
1321
1322
1323
1324
1325
1326
1327
1328
1329
1330
1331
1332
1333
1334
1335
1336
1337
1338
1339
1340
1341
1342
1343
1344
1345
1346
1347
1348
1349
1350
1351
1352
1353
1354
1355
1356
1357
1358
1359
1360
1361
1362
1363
1364
1365
1366
1367
1368
1369
1370
1371
1372
1373
1374
1375
1376
1377
1378
1379
1380
1381
1382
1383
1384
1385
1386
1387
1388
1389
1390
1391
1392
1393
1394
1395
1396
1397
1398
1399
1400
1401
1402
1403
1404
1405
1406
1407
1408
1409
1410
1411
1412
1413
1414
1415
1416
1417
1418
1419
1420
1421
1422
1423
1424
1425
1426
1427
1428
1429
1430
1431
1432
1433
1434
1435
1436
1437
1438
1439
1440
1441
1442
1443
1444
1445
1446
1447
1448
1449
1450
1451
1452
1453
1454
1455
1456
1457
1458
1459
1460
1461
1462
1463
1464
1465
1466
1467
1468
1469
1470
1471
1472
1473
1474
1475
1476
1477
1478
1479
1480
1481
1482
1483
1484
1485
1486
1487
1488
1489
1490
1491
1492
1493
1494
1495
1496
1497
1498
1499
1500
1501
1502
1503
1504
1505
1506
1507
1508
1509
1510
1511
1512
1513
1514
1515
1516
1517
1518
1519
1520
1521
1522
1523
1524
1525
1526
1527
1528
1529
1530
1531
1532
1533
1534
1535
1536
1537
1538
1539
1540
1541
1542
1543
1544
1545
1546
1547
1548
1549
1550
1551
1552
1553
1554
1555
1556
1557
1558
1559
1560
1561
1562
1563
1564
1565
1566
1567
1568
1569
1570
1571
1572
1573
1574
1575
1576
1577
1578
1579
1580
1581
1582
1583
1584
1585
1586
1587
1588
1589
1590
1591
1592
1593
1594
1595
1596
1597
1598
1599
1600
1601
1602
1603
1604
1605
1606
1607
1608
1609
1610
1611
1612
1613
1614
1615
1616
1617
1618
1619
1620
1621
1622
1623
1624
1625
1626
1627
1628
1629
1630
1631
1632
1633
1634
1635
1636
1637
1638
1639
1640
1641
1642
1643
1644
1645
1646
1647
1648
1649
1650
1651
1652
1653
1654
1655
1656
1657
1658
1659
1660
1661
1662
1663
1664
1665
1666
1667
1668
1669
1670
1671
1672
1673
1674
1675
1676
1677
1678
1679
1680
1681
1682
1683
1684
1685
1686
1687
1688
1689
1690
1691
1692
1693
1694
1695
1696
1697
1698
1699
1700
1701
1702
1703
1704
1705
1706
1707
1708
1709
1710
1711
1712
1713
1714
1715
1716
1717
1718
1719
1720
1721
1722
1723
1724
1725
1726
1727
1728
1729
1730
1731
1732
1733
1734
1735
1736
1737
1738
1739
1740
1741
1742
1743
1744
1745
1746
1747
1748
1749
1750
1751
1752
1753
1754
1755
1756
1757
1758
1759
1760
1761
1762
1763
1764
1765
1766
1767
1768
1769
1770
1771
1772
1773
1774
1775
1776
1777
1778
1779
1780
1781
1782
1783
1784
1785
1786
1787
1788
1789
1790
1791
1792
1793
1794
1795
1796
1797
1798
1799
1800
1801
1802
1803
1804
1805
1806
1807
1808
1809
1810
1811
1812
1813
1814
1815
1816
1817
1818
1819
1820
1821
1822
1823
1824
1825
1826
1827
1828
1829
1830
1831
1832
1833
1834
1835
1836
1837
1838
1839
1840
1841
1842
1843
1844
1845
1846
1847
1848
1849
1850
1851
1852
1853
1854
1855
1856
1857
1858
1859
1860
1861
1862
1863
1864
1865
1866
1867
1868
1869
1870
1871
1872
1873
1874
1875
1876
1877
1878
1879
1880
1881
1882
1883
1884
1885
1886
1887
1888
1889
1890
1891
1892
1893
1894
1895
1896
1897
1898
1899
1900
1901
1902
1903
1904
1905
1906
1907
1908
1909
1910
1911
1912
1913
1914
1915
1916
1917
1918
1919
1920
1921
1922
1923
1924
1925
1926
1927
1928
1929
1930
1931
1932
1933
1934
1935
1936
1937
1938
1939
1940
1941
1942
1943
1944
1945
1946
1947
1948
1949
1950
1951
1952
1953
1954
1955
1956
1957
1958
1959
1960
1961
1962
1963
1964
1965
1966
1967
1968
1969
1970
1971
1972
1973
1974
1975
1976
1977
1978
1979
1980
1981
1982
1983
1984
1985
1986
1987
1988
1989
1990
1991
1992
1993
1994
1995
1996
1997
1998
1999
2000
2001
2002
2003
2004
2005
2006
2007
2008
2009
2010
2011
2012
2013
2014
2015
2016
2017
2018
2019
2020
2021
2022
2023
2024
2025
2026
2027
2028
2029
2030
2031
2032
2033
2034
2035
2036
2037
2038
2039
2040
2041
2042
2043
2044
2045
2046
2047
2048
2049
2050
2051
2052
2053
2054
2055
2056
2057
2058
2059
2060
2061
2062
2063
2064
2065
2066
2067
2068
2069
2070
2071
2072
2073
2074
2075
2076
2077
2078
2079
2080
2081
2082
2083
2084
2085
2086
2087
2088
2089
2090
2091
2092
2093
2094
2095
2096
2097
2098
2099
2100
2101
2102
2103
2104
2105
2106
2107
2108
2109
2110
2111
2112
2113
2114
2115
2116
2117
2118
2119
2120
2121
2122
2123
2124
2125
2126
2127
2128
2129
2130
2131
2132
2133
2134
2135
2136
2137
2138
2139
2140
2141
2142
2143
2144
2145
2146
2147
2148
2149
2150
2151
2152
2153
2154
2155
2156
2157
2158
2159
2160
2161
2162
2163
2164
2165
2166
2167
2168
2169
2170
2171
2172
2173
2174
2175
2176
2177
2178
2179
2180
2181
2182
2183
2184
2185
2186
2187
2188
2189
2190
2191
2192
2193
2194
2195
2196
2197
2198
2199
2200
2201
2202
2203
2204
2205
2206
2207
2208
2209
2210
2211
2212
2213
2214
2215
2216
2217
2218
2219
2220
2221
2222
2223
2224
2225
2226
2227
2228
2229
2230
2231
2232
2233
2234
2235
2236
2237
2238
2239
2240
2241
2242
2243
2244
2245
2246
2247
2248
2249
2250
2251
2252
2253
2254
2255
2256
2257
2258
2259
2260
2261
2262
2263
2264
2265
2266
2267
2268
2269
2270
2271
2272
2273
2274
2275
2276
2277
2278
2279
2280
2281
2282
2283
2284
2285
2286
2287
2288
2289
2290
2291
2292
2293
2294
2295
2296
2297
2298
2299
2300
2301
2302
2303
2304
2305
2306
2307
2308
2309
2310
2311
2312
2313
2314
2315
2316
2317
2318
2319
2320
2321
2322
2323
2324
2325
2326
2327
2328
2329
2330
2331
2332
2333
2334
2335
2336
2337
2338
2339
2340
2341
2342
2343
2344
2345
2346
2347
2348
2349
2350
2351
2352
2353
2354
2355
2356
2357
2358
2359
2360
2361
2362
2363
2364
2365
2366
2367
2368
2369
2370
2371
2372
2373
2374
2375
2376
2377
2378
2379
2380
2381
2382
2383
2384
2385
2386
2387
2388
2389
2390
2391
2392
2393
2394
2395
2396
2397
2398
2399
2400
2401
2402
2403
2404
2405
2406
2407
2408
2409
2410
2411
2412
2413
2414
2415
2416
2417
2418
2419
2420
2421
2422
2423
2424
2425
2426
2427
2428
2429
2430
2431
2432
2433
2434
2435
2436
2437
2438
2439
2440
2441
2442
2443
2444
2445
2446
2447
2448
2449
2450
2451
2452
2453
2454
2455
2456
2457
2458
2459
2460
2461
2462
2463
2464
2465
2466
2467
2468
2469
2470
2471
2472
2473
2474
2475
2476
2477
2478
2479
2480
2481
2482
2483
2484
2485
2486
2487
2488
2489
2490
2491
2492
2493
2494
2495
2496
2497
2498
2499
2500
2501
2502
2503
2504
2505
2506
2507
2508
2509
2510
2511
2512
2513
2514
2515
2516
2517
2518
2519
2520
2521
2522
2523
2524
2525
2526
2527
2528
2529
2530
2531
2532
2533
2534
2535
2536
2537
2538
2539
2540
2541
2542
2543
2544
2545
2546
2547
2548
2549
2550
2551
2552
2553
2554
2555
2556
2557
2558
2559
2560
2561
2562
2563
2564
2565
2566
2567
2568
2569
2570
2571
2572
2573
2574
2575
2576
2577
2578
2579
2580
2581
2582
2583
2584
2585
2586
2587
2588
2589
2590
2591
2592
2593
2594
2595
2596
2597
2598
2599
2600
2601
2602
2603
2604
2605
2606
2607
2608
```



```
A = LOAD '/home/cloudera/java.txt' USING PigStorage(',') AS
(a1:int,a2:chararray,a3:chararray,a4:chararray) ;
--execute and verify data
dump A;
```

```
-----
-- get storage structure
describe A ;
```

```
-----
---cross join
A = LOAD '/home/cloudera/M.txt' USING PigStorage(',') AS (a1:int,a2:int)
; dump A;

B = LOAD '/home/cloudera/N.txt' USING PigStorage(',') AS (a1:int,a2:int) ;

dump B;
```

```
result = cross A,B;
```

```
-- distinct
```

```
A = LOAD '/home/cloudera/M.txt' USING PigStorage(',') AS (a1:int,a2:int)
```

```
; dump A;
```

```
Result = DISTINCT A;
```

```
dump Result
```

```
----filter command
```

```
A = LOAD '/home/cloudera/java.txt' USING PigStorage(',') AS  
(a1:int,a2:chararray,a3:chararray,a4:chararray) ;
```

```
dump A
```

```
result = filter A by a1>9
```

```
dump Result
```

```
---Foreach command
```

```
A = LOAD '/home/cloudera/java.txt' USING PigStorage(',') AS  
(a1:int,a2:chararray,a3:chararray,a4:chararray) ;
```

```
dump A
```

```
result= foreach A generate a1*10,a2;
```

```
dump Result;
```

```
---group command
```

```
A = LOAD '/home/cloudera/alldata.txt' USING PigStorage(',') AS  
(a1:int,a2:chararray,a3:chararray,a4:chararray) ;
```

```
dump A
```

Name of the Laboratory: _____

Experiment No: _____ Date: _____

result = group A by a4 ;

dump result;

-cogroup command

A = LOAD '/home/cloudera/java.txt' USING PigStorage(',') AS
(a1:int,a2:chararray,a3:chararray,a4:chararray) ;

dump A

B = LOAD '/home/cloudera/Hadoop' USING PigStorage(',') AS
(a1:int,a2:chararray,a3:chararray,a4:chararray) ;

dump B

CG= cogroup A by a4, B by a4

---Limit command -- limit no of rows

result = LIMIT B

2; dump result;

---order/ sorting

command result = order

B by a4 desc dump

result;

--split command

SPLIT A INTO X IF a1<=2, Y IF a1>2;

dum

p X;

Name of the Laboratory: _____

Experiment No: _____ Date: _____

dum

p Y;

--union

command Z =

union X,Y

dump Z

empcode int

empfname string

emplname

string

job string

manager string

hiredate string

salary int

commission int

deptcode int

---Aggrigate functions:

```
A = LOAD '/home/cloudera/employee_data.txt' USING PigStorage(',') AS  
(empcode:int,empfname:chararray,emplname:chararray,job:chararray,  
manager:chararray, hiredate:chararray,salary:int, commission:int, deptcode:int);
```

Name of the Laboratory: _____**Experiment No:** _____ **Date:** _____

dump A;

B = group A by

job; dump B;

C= foreach B generate group,

SUM(A.salary); dump C;

--string function concat

A = LOAD '/home/cloudera/employee_data.txt' USING PigStorage(',') AS
(empcode:int,empfname:chararray,emplname:chararray,job:chararray,
manager:chararray,hiredate:chararray,salary:int,commission:int,deptcode:int);

dump A;

C= foreach A generate CONCAT(A.empfname,A.emplname),

A.salary; dump C;

--mathamatical function

A = LOAD '/home/cloudera/employee_data.txt' USING PigStorage(',') AS
(empcode:int,empfname:chararray,emplname:chararray,job:chararray,
manager:chararray,hiredate:chararray,salary:int,commission:int,deptcode:int);

dump A;

B = group A by

job; dump B;

C= foreach B generate A.job,

FLOOR(AVG(A.salary)); dump C;

C= foreach B generate A.job, FLOOR(AVG(A.salary));

---self join

```
A = LOAD '/home/cloudera/employee_data.txt' USING PigStorage(',') AS  
(empcode:int,empfname:chararray,emplname:chararray,job:chararray,  
manager:chararray,hiredate:chararray,salary:int,commission:int,deptcode:int);
```

```
dump A;
```

```
B = LOAD '/home/cloudera/dept_data.txt' USING PigStorage(',') AS (deptcode:int,  
deptname:chararray,location:chararray);
```

```
dump B;
```

```
dept3 = JOIN A BY deptcode, B BY
```

```
deptcode; dump dept3
```

```
dept3 = JOIN A BY deptcode, B BY
```

```
deptcode; dump dept 3
```

```
dept3 = JOIN A BY deptcode LEFT OUTER, B BY
```

```
deptcode; dump dept 3
```

```
dept3 = JOIN A BY deptcode RIGHT OUTER, B BY
```

```
deptcode; dump dept 3
```

```
dept3 = JOIN A BY deptcode FULL OUTER, B BY
```

```
deptcode; dump dept 3
```

--- Sum of two numbers;

```
A = LOAD '/home/cloudera/employee_data.txt' USING PigStorage(',') AS  
(empcode:int,empfname:chararray,emplname:chararray,job:chararray,  
manager:chararray,hiredate:chararray,salary:int,commission:int,deptcode:int);
```

```
dump A;
```

Name of the Laboratory: _____**Experiment No:** _____ **Date:** _____

B= foreach A generate salary+commission as TotalSal,salary,commission

C= foreach A generate empcode, salary, empfname, emplname, salary+empcode

---- How to run .pig file

grunt> exec /home/cloudera/wordcount.pig

word count

input2 = LOAD '/home/cloudera/wordcount.txt' AS (line:Chararray);

Words = FOREACH input2 GENERATE FLATTEN(TOKENIZE(line,' '))

AS word;

Grouped = GROUP Words BY word;

wordcount = FOREACH Grouped GENERATE COUNT(Words),group;

dump wordcount;

—department wise maximum salary

A = LOAD '/home/cloudera/employee_data.txt' USING PigStorage(',') AS
(empcode:int,empfname:chararray,emplname:chararray,job:chararray,
manager:chararray,hiredate:chararray,salary:int, commission:int, deptcode:int);

dump A;

B = group A by deptcode

dump B;

C = foreach B generate group, MAX(salary);

dump C;


```
A = LOAD '/home/cloudera/employee_data.txt' USING PigStorage(',') AS  
(empcode:int,empfname:chararray,emplname:chararray,job:chararray,  
manager:chararray, hiredate:chararray,salary:int, commission:int, deptcode:int);
```

```
dump A;
```

```
B = order A by job;
```

```
C = foreach B generate empcode, concat(empfname+emplname), substr(job,1,3)
```

```
dump C;
```

```
C = foreach A generate CONCAT(empfname,emplname),  
SUBSTRING(job,1,3),empcode,hiredate,salary;
```

Experiment 5**4.SQOOP**

I. Create table in HIVE using hive query language. [cloudera@quickstart ~]\$ hive

hive> show

databases;

O/P:-

OK

hive> use

default;

O/P:-

OK

Aks

Aksra

ara

default

Time taken: 0.026 seconds

hive> show tables;

O/P:- OK

my_table

user_data

user_data1

hive> CREATE TABLE my_table (

> id INT,

> name STRING,

> age INT

>)

> ROW FORMAT DELIMITED

Name of the Laboratory: _____**Experiment No:** _____ **Date:** _____

```
> FIELDS TERMINATED BY ','
> STORED AS
  TEXTFILE;
> O/P:- OK
hive> INSERT INTO my_table VALUES
  > (1, 'Alice', 25),
  > (2, 'Bob', 30),
  > (3, 'Charlie', 35),
  > (4, 'Dave', 40),
  > (5, 'Eve', 45),
  > (6, 'Frank', 50),
  > (7, 'Grace', 55),
  > (8, 'Henry', 60),
  > (9, 'Isabel', 65),
  > (10, 'John', 70);
```

> O/P:-

Loading data to table default.my_table

Table default.my_table stats: [numFiles=1, numRows=10, totalSize=108,
rawDataSize=98]

MapReduce Jobs Launched:

Stage-Stage-1: Map: 1 Cumulative CPU: 1.08 sec HDFS Read:
4115 HDFS

Write: 180 SUCCESS

Total MapReduce CPU Time Spent: 1 seconds 80 msec

OK

II. Import the sql table data into hive using sqoop too.

To activate MySQL : \$ mysql -u root -

pcloudera MySQL Database name :

retail_db

MySQL Table name : categories

\$ sqoop import \

Name of the Laboratory: _____**Experiment No:** _____ **Date:** _____

```
--connect jdbc:mysql://localhost/retail_db \  
--username root \  
--password cloudera \  
--table categories \  
--hive-import \  
--hive-table demo3
```

III. Export hive table data into local machine and into SQL.

1) Create a hive table with data (categories)

a) Use DESCRIBE FORMATTED TABLE_NAME; to check the location and field delimiters. ex: '/0001','' etc.

2) Create a MySQL table (cats) with same schema as the hive table

3) Run the following command in cloudera and check in MySQL

```
$ sqoop export --connect jdbc:mysql://localhost/mysql -m 1 --table cats –
```

```
export-dir /user/hive/warehouse/categories --input-fields-terminated-by
```

```
'\0001' --username root --password cloudera;
```

Experiment 6

Time series: when all other factors are constant prediction of future values.

- Different Methods of doing Time series Analysis and Forecasting
 - o ARIMA model.
 - o Seasonally ARIMA. Most used.
 - o Holt Winter Exponential Smoothing. Easiest and effective model. Link
 - o Advanced Models

1. Important Concepts and terminology in Time series Analysis.

- o Stationarity. To know everything follow the link.
 - A stationary time series is one whose properties (ie mean, variance, autocorrelation) does not depend on the time.
- o Autoregression AR.
- o Moving Average MA
- o Integration & Difference
- o ACF and PACF Plots
- o Time series components:
 - Trend: long term smooth movement, upward or downward
 - Seasonal: periodic fluctuation, less than 1 year, most commonly found in industry.
 - Cyclical: periodic fluctuation, more than 1 year.
 - irregularity: random movement.

ARIMA is the Most common model used for time series forecasting. It has 3 components.

- 1. Autoregression AR.**
- 2. Moving Average MA**
- 3. Integrated**

Name of the Laboratory: _____**Experiment No:** _____ **Date:** _____**1. Autoregression AR.**

- o Future values of Y is dependent of previous lagged values of Y.
- o regression of y_t on y_{t-1} , y_{t-2} .
- o $P = \text{ORDER OF AR}$; current value of y is dependent on how many previous lagged value of current Y. if $p=2$ that means y_t is dependent on y_{t-1} and y_{t-2} .
- o P from PACF o Interpretation of PACF:

2. Moving Average MA.

- Future values of Y is dependent of previous lagged values of white noise ie the irregular component. white noise is just the error. error is the difference between the actual value and predicted value. so we take into consideration the error also to predict the future value.
- o autocorrelation between the errors.
 - o Trend, s , c components of TS is captured in AR whereas the irregular comp is captured in MA.
 - o q is order of MA.
 - o ACF gives q .

3. Integrated

- o Integrated means no of times we difference the data then we have to integrated it back to get original series back.
- o We difference to remove trend and seasonality to it stationary series as only after making a series stationary we can implement AR and MA.

ARIMA and Seasonal ARIMA

Autoregressive Integrated Moving Averages

The general process for ARIMA models is the following:

- Visualize the Time Series Data
- Make the time series data stationary
- Plot the Correlation and AutoCorrelation Charts
- Construct the ARIMA Model or Seasonal ARIMA based on the data
- Use the model to make predictions Let's go through these steps!

```
In [3]: import numpy as np
import pandas as pd

import matplotlib.pyplot as plt
%matplotlib inline
```

```
In [4]: df=pd.read_csv('perrin-freres-monthly-champagne-.csv')
```

```
In [5]: df.head()
```

```
Out[5]:      Month  Perrin Freres monthly champagne sales millions 764-772
```

0	1964-01	2815.0
1	1964-02	2672.0
2	1964-03	2755.0
3	1964-04	2721.0
4	1964-05	2946.0

```
In [6]:
```

```
df.tail()
```

```
Out[6]:
```

	Month	Perrin Freres monthly champagne sales millions 764-772
102	1972-07	4298.0
103	1972-08	1413.0

```
In [7]: ## Cleaning up the data
df.columns=["Month","Sales"]
df.head()
```

```
Out[7]:
```

	Month	Sales
0	1964-01	2815.0
1	1964-02	2672.0
2	1964-03	2755.0
3	1964-04	2721.0
4	1964-05	2946.0

```
In [9]:
```

```
In [8]: ## Drop Last 2 rows
df.drop(106,axis=0,inplace=True)
```

```
df.tail()
```

```
Out[9]:
```

	Month	Sales
101	1972-06	5312.0
102	1972-07	4298.0
103	1972-08	413.0
104	1972-09	5877.0
105	NaN	NaN

```
In [11]:
```

```
In [10]: df.drop(105,axis=0,inplace=True)
```

```
df.tail()
```

```
Out[11]:
```

	Month	Sales
100	1972-05	4618.0
101	1972-06	5312.0
102	1972-07	4298.0
103	1972-08	413.0
104	1972-09	5877.0

```
In [12]: # Convert Month into Datetime
df['Month']=pd.to_datetime(df['Month'])
```


Name of the Laboratory: _____

Experiment No: _____ Date: _____

```
0    1964-01-012815.0
1    1964-02-012672.0
2    1964-03-012755.0
3    1964-04-012721.0
4    1964-05-012946.0
```

In [15]:

```
In [14]: df.set_index('Month',inplace=True)
```

df.head()

Out[15]:

	Sales
Month	
1964-01-01	2815.0
1964-02-01	2672.0
1964-03-01	2755.0
1964-04-01	2721.0
1964-05-01	2946.0

In [16]:

df.describe()

Out[16]:

	Sales
count	105.000000
mean	4761.152381
std	2553.502601
min	1413.000000
25%	3113.000000
50%	4217.000000
75%	5221.000000
max	13916.000000

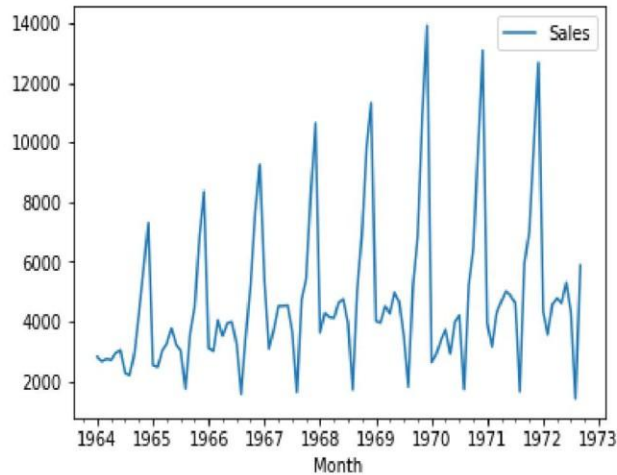
Step 2: Visualize the Data

Name of the Laboratory: _____

Experiment No: _____ Date: _____

```
In
[17]:
df.plot()
```

Out[17]: <matplotlib.axes._subplots.AxesSubplot at 0x1d2c881a2e8>



```
In [18]: ### Testing For Stationarity
```

```
from statsmodels.tsa.stattools import adfuller
```

```
In [19]: test_result=adfuller(df['Sales'])
```

```
In [20]: #Ho: It is non stationary
```

```
#H1: It is stationary
```

```
def adfuller_test(sales):
    result=adfuller(sales)
    labels = ['ADF Test Statistic', 'p-value', '#Lags Used', 'Number of Observati
    for value,label in zip(result,labels):
        print(label+' : '+str(value) )
    if result[1] <= 0.05:
        print("strong evidence against the null hypothesis(Ho), reject the nul
    else:
        print("weak evidence against null hypothesis, time series has a unit r
```

```
In [21]: adfuller_test(df['Sales'])
```

ADF Test Statistic : -1.8335930563276297

p-value : 0.3639157716602417

#Lags Used : 11

Number of Observations Used : 93

weak evidence against null hypothesis, time series has a unit root, indicating it is non-stationary

Name of the Laboratory: _____

Experiment No: _____ Date: _____

Differencing

```
In [22]: df['Sales First Difference'] = df['Sales'] - df['Sales'].shift(1)
In [212]: df['Sales'].shift(1)
```

```
Out[212]: Month
          1964-01-01      NaN
          1964-02-01    2815.0
          1964-03-01    2672.0
          1964-04-01    2755.0
          1964-05-01    2721.0
          1964-06-01    2946.0
          1964-07-01    3036.0
          1964-08-01    2282.0
          1964-09-01    2212.0
          1964-10-01    2922.0
          1964-11-01    4301.0
          1964-12-01    5764.0
          1965-01-01    7312.0
          1965-02-01    2541.0
          1965-03-01    2475.0
          1965-04-01    3031.0
          1965-05-01    3266.0
          1965-06-01    3776.0
          1965-07-01    3230.0
          1965-08-01    3028.0
          1965-09-01    1759.0
          1965-10-01    3595.0
          1965-11-01    4474.0
          1965-12-01    6838.0
          1966-01-01    8357.0
          1966-02-01    3113.0
          1966-03-01    3006.0
          1966-04-01    4047.0
          1966-05-01    3523.0
          1966-06-01    3937.0
          ...
          1970-04-01    3370.0
          1970-05-01    3740.0
          1970-06-01    2927.0
          1970-07-01    3986.0
          1970-08-01    4217.0
          1970-09-01    1738.0
          1970-10-01    5221.0
          1970-11-01    6424.0
          1970-12-01    9842.0
```

Name of the Laboratory: _____

Experiment No: _____ Date: _____

```

1971-07-01    4874.0
1971-08-01    4633.0
1971-09-01    1659.0
1971-10-01    5951.0
1971-11-01    6981.0
1971-12-01    9851.0
1972-01-01   12670.0
1972-02-01    4348.0
1972-03-01    3564.0
1972-04-01    4577.0
1972-05-01    4788.0
1972-06-01    4618.0
1972-07-01    5312.0
1972-08-01    4298.0
1972-09-01    1413.0

```

Name: Sales, Length: 105, dtype: float64

In [190]:

In [188]: `df['Seasonal First Difference']=df['Sales']-df['Sales'].shift(12)`

`df.head(14)`

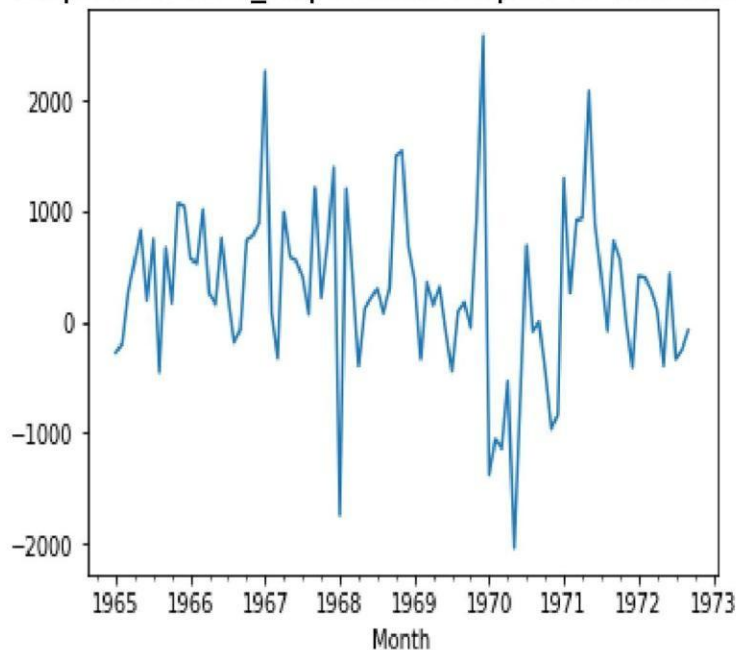
Out[190]:

	Sales	Sales First Difference	forecast	Seasonal First Difference	Month
--	-------	------------------------	----------	---------------------------	-------

1964-01-01	2815.0		NaN	NaN	NaN
1964-02-01	2672.0		-143.0	NaN	NaN
1964-03-01	2755.0		83.0	NaN	NaN
1964-04-01	2721.0		-34.0	NaN	NaN
1964-05-01	2946.0		225.0	NaN	NaN
1964-06-01	3036.0		90.0	NaN	NaN
1964-07-01	2282.0		-754.0	NaN	NaN
1964-08-01	2212.0		-70.0	NaN	NaN
1964-09-01	2922.0		710.0	NaN	NaN
1964-10-01	4301.0		1379.0	NaN	NaN
1964-11-01	5764.0		1463.0	NaN	NaN
1964-12-01	7312.0	1548.0	NaN	NaN	1965-01-01 2541.0 -4771.0 NaN -274.0
1965-02-01	2475.0		-66.0	NaN	-197.0

```
In [193]: df['Seasonal First Difference'].plot()
```

```
Out[193]: <matplotlib.axes._subplots.AxesSubplot at 0x1d2da817e80>
```



Auto Regressive Model

$$y_t = c + \phi_1 y_{t-1} + \phi_2 y_{t-2} + \cdots + \phi_p y_{t-p} + \varepsilon_t,$$

```
In [54]: from pandas.tools.plotting import autocorrelation_plot  
autocorrelation_plot(df['Sales'])  
plt.show()
```

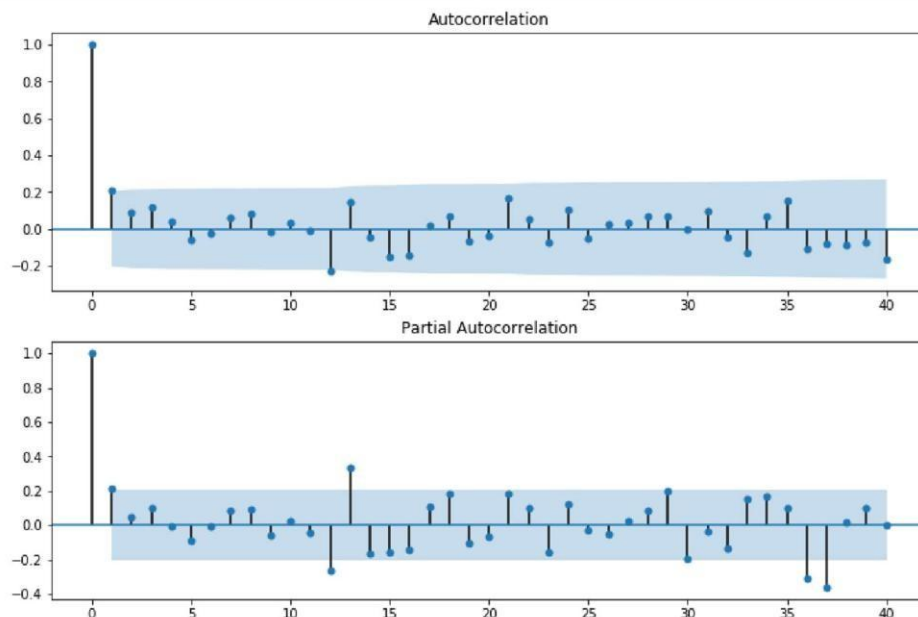
C:\Users\krish.naik\AppData\Local\Continuum\anaconda3\lib\site-packages\ipykernel_launcher.py:2: FutureWarning:
'pandas.tools.plotting.autocorrelation_plot' is deprecated, import
'pandas.plotting.autocorrelation_plot' instead.

- For an AR model, the theoretical PACF “shuts off” past the order of the model. The phrase “shuts off” means that in theory the partial autocorrelations are equal to 0 beyond that point. Put another way, the number of non-zero partial autocorrelations gives the order of the AR model. By the “order of the model” we mean the most extreme lag of x that is used as a predictor.
- Identification of an MA model is often best done with the ACF rather than the PACF.
 - For an MA model, the theoretical PACF does not shut off, but instead tapers toward 0 in some manner. A clearer pattern for an MA model is in the ACF. The ACF will have non-zero autocorrelations only at lags involved in the model.

p,d,q p AR model lags d differencing q MA lags

```
In [26]: from statsmodels.graphics.tsaplots import plot_acf, plot_pacf
```

```
In [203]: fig = plt.figure(figsize=(12,8))
ax1 = fig.add_subplot(211)
fig = sm.graphics.tsa.plot_acf(df['Seasonal First Difference'].iloc[13:],lags=
ax2 = fig.add_subplot(212)
fig = sm.graphics.tsa.plot_pacf(df['Seasonal First Difference'].iloc[13:],lags
```



```
In [115]: # For non-seasonal data
#p=1, d=1, q=0 or 1
from statsmodels.tsa.arima_model import ARIMA

In [176]: model=ARIMA(df['Sales'],order=(1,1,1))
model_fit=model.fit()
```

```
C:\Users\krish.naik\AppData\Local\Continuum\anaconda3\lib\site-packages\statsmodels\tsa\base\tsa_model.py:171: ValueWarning: No frequency information was provided, so inferred frequency MS will be used.
```

```
% freq, ValueWarning)
```

```
C:\Users\krish.naik\AppData\Local\Continuum\anaconda3\lib\site-packages\statsmodels\tsa\base\tsa_model.py:171: ValueWarning: No frequency information was provided, so inferred frequency MS will be used. % freq, ValueWarning)
```

In [177]: `model_fit.summary()`

Out[177]:

ARIMA Model Results

Dep. Variable:	D.Sales	No. Observations:	104
Model:	ARIMA(1, 1, 1)	Log Likelihood	-951.126
Method:	csm-mle	S.D. of innovations	2227.262
Date:	Wed, 18 Mar 2020	AIC	1910.251
Time:	13:40:32	BIC	1920.829
Sample:	02-01-1964	HQIC	1914.536
	- 09-01-1972		

	coef	std err	z	P> z	[0.025	0.975]
const	22.7838	12.405	1.837	0.069	-1.530	47.098
ar.L1.D.Sales	0.4343	0.089	4.866	0.000	0.259	0.609
ma.L1.D.Sales	-1.0000	0.026	-38.503	0.000	-1.051	-0.949

Roots

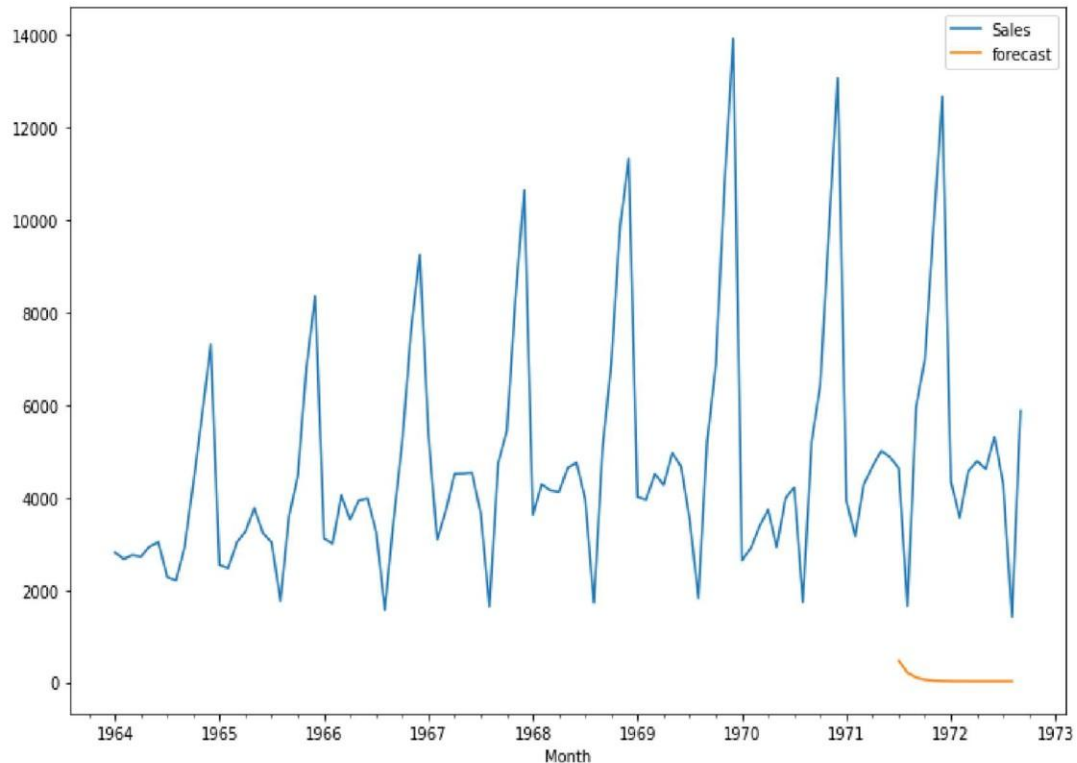
	Real	Imaginary	Modulus	Frequency
AR.1	2.3023	+0.0000j	2.3023	0.0000
MA.1	1.0000	+0.0000j	1.0000	0.0000

Name of the Laboratory: _____

Experiment No: _____ Date: _____

```
In [178]: df['forecast']=model_fit.predict(start=90,end=103,dynamic=True)
df[['Sales','forecast']].plot(figsize=(12,8))
```

```
Out[178]: <matplotlib.axes._subplots.AxesSubplot at 0x1d2d9f1fda0>
```



```
In [113]: import statsmodels.api as sm
```

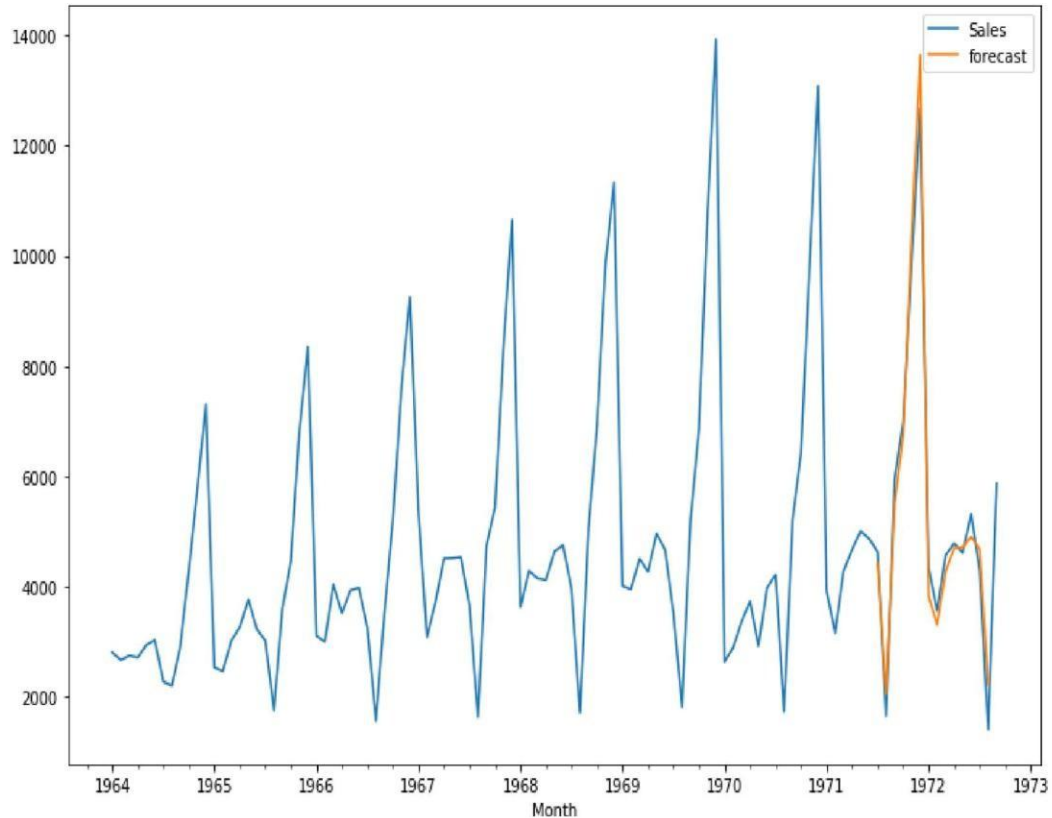
```
In [204]: model=sm.tsa.statespace.SARIMAX(df['Sales'],order=(1, 1, 1),seasonal_order=(1,
results=model.fit())
```

C:\Users\krish.naik\AppData\Local\Continuum\anaconda3\lib\site-packages\statsmodels\tsa\base\tsa_model.py:171: ValueWarning: No frequency information was provided, so inferred frequency MS will be used.
% freq, ValueWarning)

```
In [205]: df['forecast']=results.predict(start=90,end=103,dynamic=True)
df[['Sales','forecast']].plot(figsize=(12,8))
```

```
Out[205]: <matplotlib.axes._subplots.AxesSubplot at 0x1d2db70e278>
```

```
In [209]: future_df=pd.concat([df,future_datest_df])
```

```
In [206]: from pandas.tseries.offsets import DateOffset
future_dates=[df.index[-1]+ DateOffset(months=x) for x in range(0,24)]
```

```
In [207]: future_datest_df=pd.DataFrame(index=future_dates[1:],columns=df.columns)
```

```
In [208]: future_datest_df.tail()
```

Out[208]:

	Sales	Sales First Difference	forecast	Seasonal First Difference
1974-04-01	NaN	NaN	NaN	NaN
1974-05-01	NaN	NaN	NaN	NaN
1974-06-01	NaN	NaN	NaN	NaN
1974-07-01	NaN	NaN	NaN	NaN
1974-08-01	NaN	NaN	NaN	NaN

	Sales	Sales First Difference	forecast	Seasonal First Difference
1974-04-01	NaN	NaN	NaN	NaN
1974-05-01	NaN	NaN	NaN	NaN
1974-06-01	NaN	NaN	NaN	NaN
1974-07-01	NaN	NaN	NaN	NaN
1974-08-01	NaN	NaN	NaN	NaN

Name of the Laboratory: _____**Experiment No:** _____ **Date:** _____

In

```
[201]: future_df['forecast'] = results.predict(start = 104, end = 120,  
dynamic= True) future_df[['Sales', 'forecast']].plot(figsize=(12, 8))
```

Out[201]: <matplotlib.axes._subplots.AxesSubplot at 0x1d2daee5048>

